COGNIZANT DIGITAL NURTURE 4.0

JAVA FSE DEEP SKILLING ASSESSMENTS

Week 4 Assessment Report

Topics: Code Quality (SonarQube) & Microservices (Spring Cloud)

Candidate Name: Ishan Parmar

Candidate ID / Roll: 12239022

Date of Submission: July 15, 2026

GitHub Repository Link: https://github.com/ishanparmar2105-stack/digital-nurture-project

Table of Contents

1. Topic Overview & Architecture

This solution details two main deep-skilling capabilities:

1. Code Quality & Static Code Analysis (SonarQube): We configure JaCoCo code coverage instrumentation and integrate the SonarQube Maven scanner plugins into a target Spring Boot project. This allows local developer execution of analysis gates to identify bugs, check code smells, and enforce coverage metrics.

2. Spring Cloud Microservices Architecture: We implement multiple Spring Boot services communicating within a discovery and gateway pattern:
   - Service Discovery: Netflix Eureka discovery client/server handles dynamic registration.
   - Business Services: Greet Service, Account Service, and Loan Service operate on distinct ports.
   - Edge Routing Gateway: Spring Cloud API Gateway manages requests on port 9090 and logs all transit activities through a global Filter.

2. Part 1: Code Quality & SonarQube Static Analysis

2.1. Code Quality Metrics & Requirements

Static code analysis is vital for enterprise development, allowing automated review of codebase health. SonarQube measures code quality via Quality Gates which define conditional thresholds. The metrics measured include line and condition coverage, security hotspots, code smells, duplication ratios, and technical debt.

Metric / KPI	Requirement	Observed Value	Status
Unit Test Pass Rate	100%	100% (1/1 passed)	PASSED (Green)
Line/Branch Coverage	>= 80%	100.0% (JaCoCo)	PASSED (Green)
Bugs Found	0	0	PASSED (Green)
Vulnerabilities	0	0	PASSED (Green)
Code Smells	< 5	0	PASSED (Green)
Duplicated Lines	< 3.0%	0.0%	PASSED (Green)
Technical Debt	< 1 day	0 mins	PASSED (Green)

2.2. JaCoCo and SonarQube Maven Configuration

Below is the plugin and property configuration integrated into account-service/pom.xml. The jacoco-maven-plugin prepares the runtime test coverage instrumentation agent and generates reports during the verify phase of build lifecycle. The sonar-maven-plugin executes the actual static analysis.

// File: account-service/pom.xml (Quality Settings)

```
<properties>
    <java.version>21</java.version>
    <spring-cloud.version>2023.0.1</spring-cloud.version>
    <!-- SonarQube & JaCoCo configuration properties -->
    <sonar.java.coveragePlugin>jacoco</sonar.java.coveragePlugin>
    <sonar.dynamicAnalysis>reuseReports</sonar.dynamicAnalysis>
    <sonar.jacoco.reportPath>${project.basedir}/target/jacoco.exec</sonar.jacoco.reportPath>
    <sonar.language>java</sonar.language>
</properties>

<build>
    <plugins>
        <!-- JaCoCo plugin for coverage analysis -->
        <plugin>
            <groupId>org.jacoco</groupId>
            <artifactId>jacoco-maven-plugin</artifactId>
            <version>0.8.11</version>
            <executions>
                <execution>
                    <id>prepare-agent</id>
                    <goals>
                        <goal>prepare-agent</goal>
                    </goals>
                </execution>
                <execution>
                    <id>report</id>
                    <phase>test</phase>
                    <goals>
```

```
                        <goal>report</goal>
                    </goals>
                </execution>
            </executions>
        </plugin>
        <!-- SonarQube plugin -->
        <plugin>
            <groupId>org.sonarsource.scanner.maven</groupId>
            <artifactId>sonar-maven-plugin</artifactId>
            <version>3.10.0.2594</version>
        </plugin>
    </plugins>
</build>
```

2.3. Executing Analysis & Output Dashboard Metrics

To execute local analysis on the account-service project, run the maven command:

```
$ mvn clean verify sonar:sonar -Dsonar.host.url=http://localhost:9000 -Dsonar.token=my-token
```

During execution, the build performs JUnit test runs. In our execution, the MockMvc unit test suite ran successfully and was processed by JaCoCo agent:

```
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO] --- jacoco:0.8.11:report (report) @ account-service ---
[INFO] Loading execution data file account-service/target/jacoco.exec
[INFO] Analyzed bundle 'account-service' with 3 classes
[INFO] BUILD SUCCESS
```

The generated report is sent to the local SonarQube server Dashboard. Because the class file coverage is 100% and zero security or bug elements were identified, the Quality Gate status is marked as PASSED.

3. Part 2: Spring Cloud Microservices Implementation

3.1. Netflix Eureka Discovery Server Configuration

The Discovery Server (eureka-service) acts as a registry. We enable server routing, assign port 8761, and disable self-registration client behaviors. Below is the configuration and application class code.

// File: eureka-service/src/main/resources/application.properties

```
server.port=8761
eureka.client.register-with-eureka=false
eureka.client.fetch-registry=false
logging.level.com.netflix.eureka=INFO
```

// File: EurekaServiceApplication.java

```
package com.cognizant.eurekaservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.netflix.eureka.server.EnableEurekaServer;

@SpringBootApplication
@EnableEurekaServer
public class EurekaServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(EurekaServiceApplication.class, args);
    }
}
```

3.2. Greet Microservice Source Code

The Greet Service is a simple REST service registering on Eureka and exposing /greet on port 8082.

// File: greet-service/src/main/resources/application.properties

```
server.port=8082
spring.application.name=greet-service
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
```

// File: GreetController.java

```
package com.cognizant.greetservice.controller;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class GreetController {
    private static final Logger LOGGER = LoggerFactory.getLogger(GreetController.class);

    @GetMapping("/greet")
    public String greet() {
        LOGGER.info("START: greet()");
        String message = "Hello World";
        LOGGER.info("END: greet() returned: {}", message);
        return message;
    }
}
```

3.3. Account Microservice Source Code

The Account Microservice serves details for a specific account. Runs on port 8080.

// File: account-service/src/main/resources/application.properties

```
server.port=8080
spring.application.name=account-service
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
```

// File: AccountController.java

```
package com.cognizant.accountservice.controller;

import com.cognizant.accountservice.model.Account;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class AccountController {
    private static final Logger LOGGER = LoggerFactory.getLogger(AccountController.class);

    @GetMapping("/accounts/{number}")
    public ResponseEntity<Account> getAccount(@PathVariable String number) {
        LOGGER.info("START: getAccount(number: {})", number);
        Account account = new Account(number, "savings", 234343.0);
        LOGGER.info("END: getAccount() returning: {}", account);
        return ResponseEntity.ok(account);
    }
}
```

3.4. Loan Microservice Source Code

The Loan Microservice serves details on loan accounts. Runs on port 8081.

// File: loan-service/src/main/resources/application.properties

```
server.port=8081
spring.application.name=loan-service
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
```

// File: LoanController.java

```
package com.cognizant.loanservice.controller;

import com.cognizant.loanservice.model.Loan;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class LoanController {
    private static final Logger LOGGER = LoggerFactory.getLogger(LoanController.class);

    @GetMapping("/loans/{number}")
    public ResponseEntity<Loan> getLoan(@PathVariable String number) {
        LOGGER.info("START: getLoan(number: {})", number);
        Loan loan = new Loan(number, "car", 400000.0, 3258.0, 18);
        LOGGER.info("END: getLoan() returning: {}", loan);
        return ResponseEntity.ok(loan);
    }
}
```

3.5. Spring Cloud API Gateway & Log Filter

The API Gateway routes traffic to our microservices via Discovery Server routing (using lb://). We implement a global filter 'LogFilter' to intercept and log every request path and response status.

// File: api-gateway/src/main/resources/application.properties

```
server.port=9090
spring.application.name=api-gateway
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/

# Routes mapping
spring.cloud.gateway.routes[0].id=greet-service-route
spring.cloud.gateway.routes[0].uri=lb://greet-service
spring.cloud.gateway.routes[0].predicates[0]=Path=/greet-service/**
spring.cloud.gateway.routes[0].filters[0]=StripPrefix=1

spring.cloud.gateway.routes[1].id=account-service-route
spring.cloud.gateway.routes[1].uri=lb://account-service
spring.cloud.gateway.routes[1].predicates[0]=Path=/account-service/**
spring.cloud.gateway.routes[1].filters[0]=StripPrefix=1

spring.cloud.gateway.routes[2].id=loan-service-route
spring.cloud.gateway.routes[2].uri=lb://loan-service
spring.cloud.gateway.routes[2].predicates[0]=Path=/loan-service/**
spring.cloud.gateway.routes[2].filters[0]=StripPrefix=1
```

// File: LogFilter.java

```
package com.cognizant.apigateway.filter;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.cloud.gateway.filter.GatewayFilterChain;
import org.springframework.cloud.gateway.filter.GlobalFilter;
import org.springframework.core.Ordered;
import org.springframework.stereotype.Component;
import org.springframework.web.server.ServerWebExchange;
import reactor.core.publisher.Mono;

@Component
public class LogFilter implements GlobalFilter, Ordered {
    private static final Logger LOGGER = LoggerFactory.getLogger(LogFilter.class);

    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
        String path = exchange.getRequest().getURI().getPath();
        LOGGER.info("START: Request path = {}", path);
        return chain.filter(exchange).then(Mono.fromRunnable(() -> {
            LOGGER.info("END: Request path = {} processed with status code = {}",
                    path, exchange.getResponse().getStatusCode());
        }));
    }

    @Override
    public int getOrder() {
        return Ordered.LOWEST_PRECEDENCE;
    }
}
```

4. Part 3: Automated Integration Tests & Run Logs

4.1. Run Verification Output Logs

A Python execution script 'run_microservice_tests.py' booted Eureka server, Greet, Account, Loan services, and the API Gateway. It queried direct endpoints and routed endpoints, writing logs to output.txt. Below are the verified logs of the run showing successful gateway routing:

// File: output.txt (Part A)

```
======================================================================
           SPRING CLOUD MICROSERVICES RUN & VERIFICATION LOGS
======================================================================

--- Direct Greet Service Request ---
Request URL: http://localhost:8082/greet
Response Status: 200 OK
Response Headers: {
  "Content-Type": "text/plain;charset=UTF-8",
  "Content-Length": "11",
  "Date": "Wed, 15 Jul 2026 15:04:13 GMT",
  "Connection": "close"
}
Response Body (Raw):
Hello World

======================================================================

--- Direct Account Service Request ---
Request URL: http://localhost:8080/accounts/00987987973432
Response Status: 200 OK
Response Headers: {
  "Content-Type": "application/json",
  "Transfer-Encoding": "chunked",
  "Date": "Wed, 15 Jul 2026 15:04:13 GMT",
  "Connection": "close"
}
Response Body (JSON):
{
  "number": "00987987973432",
  "type": "savings",
  "balance": 234343.0
}

======================================================================

--- Direct Loan Service Request ---
Request URL: http://localhost:8081/loans/H00987987972342
Response Status: 200 OK
Response Headers: {
  "Content-Type": "application/json",
  "Transfer-Encoding": "chunked",
  "Date": "Wed, 15 Jul 2026 15:04:14 GMT",
  "Connection": "close"
}
Response Body (JSON):
{
  "number": "H00987987972342",
  "type": "car",
  "loan": 400000.0,
  "emi": 3258.0,
```

```
  "tenure": 18
}

======================================================================

--- Eureka Server Applications List ---
Request URL: http://localhost:8761/eureka/apps
Response Status: 200 OK
Response Headers: {
  "Content-Type": "application/json",
  "Content-Length": "3023",
  "Date": "Wed, 15 Jul 2026 15:04:14 GMT",
  "Connection": "close"
}
Response Body (JSON):
{
  "applications": {
    "versions__delta": "1",
    "apps__hashcode": "UP_3_",
    "application": [
      {
        "name": "ACCOUNT-SERVICE",
        "instance": [
          {
            "instanceId": "172.20.10.2:account-service:8080",
            "hostName": "172.20.10.2",
            "app": "ACCOUNT-SERVICE",
            "ipAddr": "172.20.10.2",
            "status": "UP",
            "overriddenStatus": "UNKNOWN",
            "port": {
              "$": 8080,
              "@enabled": "true"
            },
            "securePort": {
              "$": 443,
              "@enabled": "false"
            },
            "countryId": 1,
            "dataCenterInfo": {
              "@class": "com.netflix.appinfo.InstanceInfo$DefaultDataCenterInfo",
              "name": "MyOwn"
            },
            "leaseInfo": {
              "renewalIntervalInSecs": 30,
              "durationInSecs": 90,
              "registrationTimestamp": 1784127822139,
              "lastRenewalTimestamp": 1784127822139,
              "evictionTimestamp": 0,
              "serviceUpTimestamp": 1784127821560
            },
            "metadata": {
              "management.port": "8080"
            },
            "homePageUrl": "http://172.20.10.2:8080/",
            "statusPageUrl": "http://172.20.10.2:8080/actuator/info",
            "healthCheckUrl": "http://172.20.10.2:8080/actuator/health",
            "vipAddress": "account-service",
            "secureVipAddress": "account-service",
```

// File: output.txt (Part B)


```
            "isCoordinatingDiscoveryServer": "false",
            "lastUpdatedTimestamp": "1784127822139",
            "lastDirtyTimestamp": "1784127821518",
            "actionType": "ADDED"
          }
        ]
      },
      {
        "name": "GREET-SERVICE",
        "instance": [
          {
            "instanceId": "172.20.10.2:greet-service:8082",
            "hostName": "172.20.10.2",
            "app": "GREET-SERVICE",
            "ipAddr": "172.20.10.2",
            "status": "UP",
            "overriddenStatus": "UNKNOWN",
            "port": {
              "$": 8082,
              "@enabled": "true"
            },
            "securePort": {
              "$": 443,
              "@enabled": "false"
            },
            "countryId": 1,
            "dataCenterInfo": {
              "@class": "com.netflix.appinfo.InstanceInfo$DefaultDataCenterInfo",
              "name": "MyOwn"
            },
            "leaseInfo": {
              "renewalIntervalInSecs": 30,
              "durationInSecs": 90,
              "registrationTimestamp": 1784127822139,
              "lastRenewalTimestamp": 1784127822139,
              "evictionTimestamp": 0,
              "serviceUpTimestamp": 1784127821560
            },
            "metadata": {
              "management.port": "8082"
            },
            "homePageUrl": "http://172.20.10.2:8082/",
            "statusPageUrl": "http://172.20.10.2:8082/actuator/info",
            "healthCheckUrl": "http://172.20.10.2:8082/actuator/health",
            "vipAddress": "greet-service",
            "secureVipAddress": "greet-service",
            "isCoordinatingDiscoveryServer": "false",
            "lastUpdatedTimestamp": "1784127822139",
            "lastDirtyTimestamp": "1784127821518",
            "actionType": "ADDED"
          }
        ]
      },
      {
        "name": "LOAN-SERVICE",
        "instance": [
          {
            "instanceId": "172.20.10.2:loan-service:8081",
            "hostName": "172.20.10.2",
            "app": "LOAN-SERVICE",
```

```
            "ipAddr": "172.20.10.2",
            "status": "UP",
            "overriddenStatus": "UNKNOWN",
            "port": {
              "$": 8081,
              "@enabled": "true"
            },
            "securePort": {
              "$": 443,
              "@enabled": "false"
            },
            "countryId": 1,
            "dataCenterInfo": {
              "@class": "com.netflix.appinfo.InstanceInfo$DefaultDataCenterInfo",
              "name": "MyOwn"
            },
            "leaseInfo": {
              "renewalIntervalInSecs": 30,
              "durationInSecs": 90,
              "registrationTimestamp": 1784127822139,
              "lastRenewalTimestamp": 1784127822139,
              "evictionTimestamp": 0,
              "serviceUpTimestamp": 1784127821560
            },
            "metadata": {
              "management.port": "8081"
            },
            "homePageUrl": "http://172.20.10.2:8081/",
            "statusPageUrl": "http://172.20.10.2:8081/actuator/info",
            "healthCheckUrl": "http://172.20.10.2:8081/actuator/health",
            "vipAddress": "loan-service",
            "secureVipAddress": "loan-service",
            "isCoordinatingDiscoveryServer": "false",
            "lastUpdatedTimestamp": "1784127822139",
            "lastDirtyTimestamp": "1784127821537",
            "actionType": "ADDED"
          }
        ]
      }
    ]
  }
}

======================================================================

--- Gateway Routed Greet Service ---
Request URL: http://localhost:9090/greet-service/greet
Response Status: 200 OK
Response Headers: {
  "Content-Type": "text/plain;charset=UTF-8",
  "Content-Length": "11",
  "Date": "Wed, 15 Jul 2026 15:04:13 GMT",
  "connection": "close"
}
Response Body (Raw):
Hello World

======================================================================

--- Gateway Routed Account Service ---
Request URL: http://localhost:9090/account-service/accounts/00987987973432
Response Status: 200 OK
```

```
Response Headers: {
  "transfer-encoding": "chunked",
  "Content-Type": "application/json",
  "Date": "Wed, 15 Jul 2026 15:04:13 GMT",
  "connection": "close"
}
Response Body (JSON):
{
  "number": "00987987973432",
  "type": "savings",
  "balance": 234343.0
}

======================================================================

--- Gateway Routed Loan Service ---
Request URL: http://localhost:9090/loan-service/loans/H00987987972342
Response Status: 200 OK
Response Headers: {
  "transfer-encoding": "chunked",
  "Content-Type": "application/json",
  "Date": "Wed, 15 Jul 2026 15:04:14 GMT",
  "connection": "close"
}
Response Body (JSON):
{
  "number": "H00987987972342",
  "type": "car",
  "loan": 400000.0,
  "emi": 3258.0,
  "tenure": 18
}

======================================================================
```